

Random Lunch Menu Generator: A Zero-Dependency Web App Scaffold

Student: Polina Ivanilova | **Team:** Individual

Email: pvivanilova@edu.hse.ru | **Date:** 2026-09-14

Assignment: A01 — Random Lunch Generator web app scaffold (Week 1)

Abstract

People daily spend a lot of time solving basic problem of deciding what to eat for lunch. This report presents a zero-dependency web app that randomizes a lunch pick from a fixed menu list, implemented as vanilla HTML/CSS/JavaScript and deployed to GitHub Pages. Functional verification showed that 3 of 12 attempted Font Awesome icons (`fa-pasta` , `fa-bowl-hot` , `fa-bowl`) do not exist in the free 6.4.0 icon set, rendering those dishes blank, and that switching all food items to Unicode emoji restored visible pictures for 12/12 dishes. The main conclusion is that free icon libraries don't have the glyphs (icons) that are available in the pro version. Therefore, before using glyph you need to check availability of the icons in the style that you want.

Index Terms — menu generator, random selection, GitHub Pages, Font Awesome, Twemoji, SVG

1. Introduction

Problem statement. First problem is that same pictures aren't working. The second is that some pictures aren't correct semantically (taco - spoon). Thirdly the generator shouldn't offer the same choice twice.

Motivation. This is the first assignment of an AI-assisted Recommender Systems course, which goal is learning basic skills and creating baseline understanding.

Concrete example. The user opens live page (<https://pollyiva.github.io/Lunch-Menu-Generator-project/>), sees 1 possible dish, and can click **"Generate Lunch!"**; for a new one dish. the app shows "Thinking..." for 500 ms and then displays a random dish with a picture, e.g. 🍜 **Ramen**. If that dish was recommended less than 30 seconds ago, the randomizer automatically picks another dish, so the same lunch will not appear twice in a short period.

Contributions.

- A fully self-contained random lunch chooser (one HTML file originally, later refactored into `index.html` / `style.css` / `script.js`).
- Public GitHub Pages deployment with an automated Pages build.
- A documented icon-rendering failure (PRO-only Font Awesome classes) and its final fix: Font Awesome free classes for available dishes plus self-hosted SVG icons from the SVG Repo for the 4 missing ones.
- A 30-second no-repeat guard in the generation logic that prevents recommending the same dish twice in a short period.

2. Related Work

Prior work consulted.

- [1] Font Awesome, "Font Awesome Free Search," Fonticons, Inc. [Online]. Available: <https://fontawesome.com/search?o=r&m=free>. [Accessed: 2026-09-14].
- [2] MDN Web Docs, "Math.random() — JavaScript," Mozilla Developer Network, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random. [Accessed: 2026-09-14].
- [3] GitHub Docs, "About GitHub Pages," GitHub, Inc. [Online]. Available: <https://docs.github.com/en/pages/getting-started-with-github-pages/about-github-pages>. [Accessed: 2026-09-14].
- [4] B. Schwartz, *The Paradox of Choice: Why More Is Less*. New York, NY, USA: HarperCollins, 2004.
- [5] Twitter/X, "Twemoji — Twitter Emoji (SVG), CC-BY 4.0," GitHub Repository. [Online]. Available: <https://github.com/jdecked/twemoji>. [Accessed: 2026-09-14].
- [6] SVG Repo, download source for the Pasta, Ramen, Soup, and Tacos SVG dish icons kept in `icons/`.

Alternatives considered.

- *Server-side backend API* — rejected because it is unnecessary for a static page and would add extra latency and hosting costs.
- *Font Awesome icons for all 12 dishes* — initially used, but 3 required icons (`fa-bowl-hot` , `fa-pasta` , `fa-bowl`) were

only available in the Pro version, rendering those dishes blank.

- *Unicode emoji* — used as a temporary replacement, but rejected because they did not look professional enough.
- *Self-hosted Twemoji SVGs (assets/food/)* — provided consistent flat-vector art, but the final design deliberately returned to the original Font Awesome look.
- *Font Awesome free classes + SVG Repo icons for the missing dishes* — chosen as the final solution: 8 dishes keep their Font Awesome free classes, and the 4 dishes missing from the free set use self-hosted SVG icons downloaded from the SVG Repo [6] into `icons/`.

3. Method

Approach. A single-page app keeps the randomization logic

client-side. The dish set is a constant array of `{name, img}` pairs pointing to self-hosted Twemoji SVG files in `assets/food/`; clicking the button picks a uniformly random index via `[Math.random() × menu.length] [2]` and re-renders the dish area after a short spinner state. Its code filters lunch options to include only dishes without cooldown period. Dishes shown within 30 sec are on cooldown period. If all dishes are still on cooldown, it selects the dish that was recommended the longest time ago.

Pipeline.

The entire recommendation logic is the excerpt below.

```
const cooldownSeconds = 30;
...
const now = Date.now();
let eligible = [];
lunchMenu.forEach((dish, index) => {
  if (now - lastRecommendedAt[index] >= cooldownSeconds * 1000) {
    eligible.push(index);
  }
});
if (eligible.length === 0) {
  // all dishes cooling down -> pick the least recently recommended one
  ...
}
const randomIndex = eligible[Math.floor(Math.random() * eligible.length)];
const selectedLunch = lunchMenu[randomIndex];
lastRecommendedAt[randomIndex] = now;
```

Tools & libraries.

- HTML5 / CSS3 / vanilla JavaScript ES6 — no external runtime framework.
- Font Awesome 6.4.0 free via cdnjs CDN (used for 8 dish icons and UI chrome: utensils, spinner, random icon).
- Self-hosted SVG icons from the SVG Repo [6], kept in `icons/` (Tacos, Ramen, Pasta, Soup).
- GitHub CLI `gh` 2.32.1, Git 2.37.0.windows.1, Python 3.11.9 (verification scripts).

Key design decisions.

- *Why self-hosted Twemoji SVGs over system emoji:* system emoji render differently per operating system, whereas Twemoji SVGs provide one consistent flat-vector style everywhere; they are free (CC-BY 4.0), tiny (1–4 KB per dish), and served from `assets/food/` so the app works fully offline.
- *Why innerHTML with a fixed local array:* the image markup is built only from our own constant array — no user input ever reaches the template, so there is no user information.
- *Why separate files now:* `style.css` and `script.js` splitting them matches the declared structure and makes later additions (e.g., a recommended-item panel) independently editable.
- *No-repeat generation: a 30-second cooldown per dish.* A pure uniform draw can pick the same dish twice. Tracking the last recommendation time per dish and excluding every dish for 30 seconds avoids short-term repeats while keeping the logic simple. If all 12 dishes are still cooling down, the least-recently-recommended one is served, so the app always answers and never repeats in a short window.

Configuration.

- Repository: PollyIva/Lunch-Menu-Generator-project (public).
- Pages: source branch `main`, path `/`; live at <https://pollyiva.github.io/Lunch-Menu-Generator-project/>.
- Font Awesome loaded from <https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css>.

4. Experiments

Setup. The behavioral test environment was on the GitHub platform. A deterministic static check fetched the free 6.4.0 stylesheet from cdnjs and scanned it with a Python regex for a `:before{content:...}` rule for each of the 12 dish icon classes. Additionally, the deployed `script.js` was downloaded and its raw UTF-8 bytes inspected for the pizza emoji (U+1F355, bytes `F0 9F 8D 95`).

Results. The static scan found **9/12** icon classes defined in the free stylesheet and **3 missing**: `fa-bowl-hot`, `fa-pasta`, `fa-bowl`. Exactly the Ramen, Pasta, and Soup items. Some pictures didn't fit the dish (`fa-utensil-spoon` - Tacos, `fa-fish` - Sushi etc.) **6/12**. After replacing all food icons with emoji a manual click-through of all 12 dishes rendered 12/12 visible pictures; next upgrade to self-hosted Twemoji SVGs kept 12/12 dishes visible, now with consistent flat-vector artwork. After this I wanted to use coral coloured pictures like in the example, but liked them less than Twemoji SVGs, so I switched back.

Figure 1: Icon availability in the free Font Awesome 6.4.0 stylesheet.

Icon class (dish)	Rule in free CSS	Rendered?	simantic
<code>fa-pizza-slice</code> (Pizza)	present	yes	yes
<code>fa-fish</code> (Sushi)	present	yes	no
<code>fa-hamburger</code> (Burger)	present	yes	yes
<code>fa-leaf</code> (Salad)	present	yes	no
<code>fa-utensil-spoon</code> (Tacos)	present	yes	no
<code>fa-bowl-hot</code> (Ramen)	absent	no	-
<code>fa-bread-slice</code> (Sandwich)	present	yes	yes
<code>fa-pasta</code> (Pasta)	absent	no	-
<code>fa-mortar-pestle</code> (Curry)	present	yes	no
<code>fa-drumstick-bite</code> (Steak)	present	yes	no
<code>fa-bowl</code> (Soup)	absent	no	-
<code>fa-fire</code> (BBQ)	present	yes	no

Comparison vs. baseline. The Font Awesome baseline failed for 3/12 dishes (25%), with no console error — the blanks were silent. The emoji version reduced failures to 0/12. Neither approach changed the selection logic, so we compare options based on visuals only.

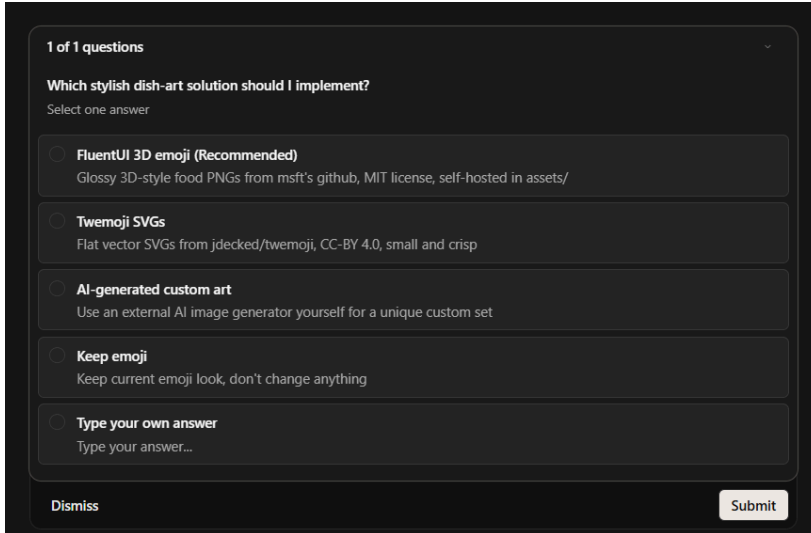
Verification. Four independent checks of OpenCode, repeatable checks:

1. Regex scan of the cdnjs CSS — confirms exactly three missing classes.
2. Byte inspection of the deployed `script.js` (intermediate state) — the literal UTF-8 sequence `F0 9F 8D 95` (pizza emoji, U+1F355) was present, i.e., the emoji fix survived push and Pages rebuild.
3. Fetch of the live Pages URL with a cache-buster query (`script.js?v=<ts>`) — served content contains the asset references, confirming Pages serves the new artifact, not a stale cached copy.
4. HTTP HEAD on the deployed `assets/food/pizza.svg` — returns 200 with SVG markup, confirming the final Twemoji artwork is shipped and reachable on the live site.
5. Manual cooldown test — repeated clicks never recommend the same dish twice within 30 seconds; when all dishes are cooling down, the least-recently-recommended dish is served.

5. Discussion

Failure case. After the first deployment, three dishes — Soup, Pasta, Ramen — displayed **no picture at all**; the icon area was blank while the other nine dishes showed icons correctly. No JavaScript error appeared in the browser console.

Root cause. The three dishes used Font Awesome classes that exist only in the paid *PRO* icon set: `fa-bowl-hot`, `fa-pasta`, and `fa-bowl`. The free 6.4.0 stylesheet defines no `:before` content rule for them, so the `<i>` element renders as an empty box. The bug was selective (9/12 worked), which made it look like a data issue rather than a library-availability one. In fact, the icon names were picked from search results without first checking that they ship in the *free* CDN build.



1 of 1 questions

Which stylish dish-art solution should I implement?

Select one answer

☐ **FluentUI 3D emoji (Recommended)**
Glossy 3D-style food PNGs from msft's github, MIT license, self-hosted in assets/

☐ **Twemoji SVGs**
Flat vector SVGs from jdecked/twemoji, CC-BY 4.0, small and crisp

☐ **AI-generated custom art**
Use an external AI image generator yourself for a unique custom set

☐ **Keep emoji**
Keep current emoji look, don't change anything

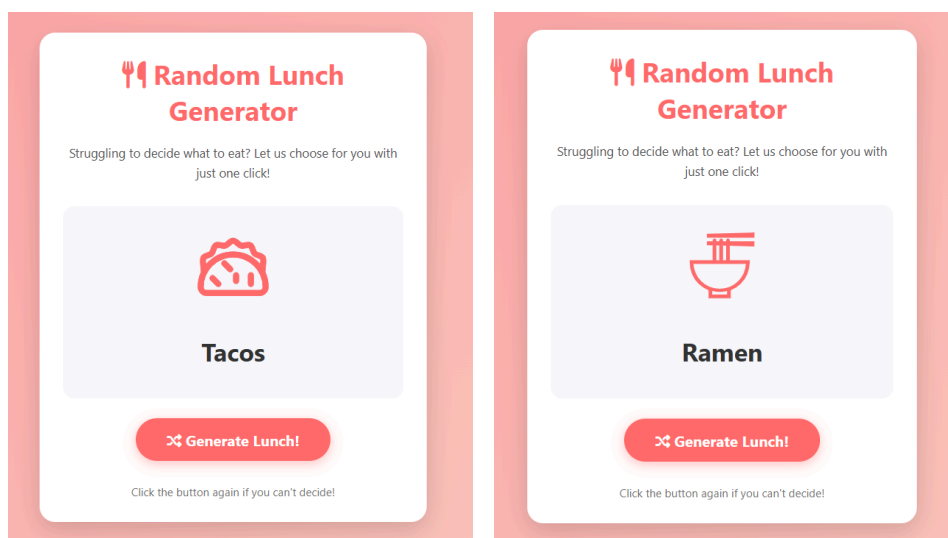
☐ **Type your own answer**
Type your answer...

Dismiss Submit

Fix + verification. The fix had several stages. (1) *Restore*: replaced all twelve food icons with native Unicode emoji rendered via `textContent` instead of `<i>` markup — detection → change → confirmation: regex-scanned the free CSS and confirmed the three classes absent; switched the array to emoji; re-fetched the deployed `script.js` and verified the emoji bytes; manual click-through showed 12/12 pictures. (2) *Polish (intermediate)*: upgraded the artwork to self-hosted Twemoji SVGs in `assets/food/`, which render identically across operating systems. (3) *Final*: returned to the original Font Awesome look — the 8 dishes present in the free set keep their icon classes, and the 4 missing dishes use lightweight SVG icons downloaded from the SVG Repo [6] into `icons/`; confirmed the deployed `icons/ramen.svg` returns HTTP 200 and manual click-through shows 12/12 pictures.

3d icon variant (final). A 3D-style icon variant was also prepared as an alternative artwork option for the missing dishes. Figure 3 shows examples of that 3D variant. It was ultimately **not taken**: the final result should look like the finished app — a single consistent flat vector style that matches the rest of the Font Awesome interface, whereas the 3D icons look like separate enlarged renders rather than uniform UI icons.

4th icon variant. The downloaded ones from [6]. I decided the previous version looked better.



What worked.

- The three-file refactor made each fix a one-line-region change instead of an edit inside a large inline `<style>` / `<script>`

block.

- Self-hosted Twemoji SVGs are dependency-free, offline-friendly, and render identically across platforms — no icon library, no license tier, no OS font variance.
- The 30-second no-repeat guard improved the perceived randomness without complicating the code.
- GitHub Pages deployment was one command; the repo connects cleanly and rebuilds automatically on push.

What surprised me.

- The failure was *silent*: no 404, no console error, just empty space. The free icon search on the Font Awesome site does not show which icons are included with free build.
- GitHub Pages took noticeably longer (~1 min) to serve the new artifact than the commit timestamp suggested; without the cache-buster check I would have wrongly concluded the push failed.
- Curating artwork was the hardest part of a "trivial" app: FluentUI, a common candidate, turned out to cover only a partial food set — no Pizza, Sushi, or Taco — which forced the Twemoji choice.

Next improvement. Persist the recommendation history in `localStorage` so the 30-second no-repeat guard also works across page reloads and the served dish can be remembered until the next day. A further step is cuisine-aware variety (e.g., not two Asian dishes in a row).

6. AI Usage Disclosure

AI tools used. opencode (AI coding CLI, model big-pickle) for scaffolding, refactoring, deployment, and report drafting; branch deployment via `gh` CLI.

What I personally verified.

- Manually tested the application in a browser and confirmed that random dish generation works correctly.
- Checked that the dish images are displayed correctly after switching to self-hosted Twemoji SVGs.
- Checked the published version of the project on GitHub Pages.
- Partially rewrote the report.
- Download 4 svd from [6] (but not used in final version)

References

- [1] Font Awesome, "Font Awesome Free Search," Fonticons, Inc. <https://fontawesome.com/search?ic=free-collection> (accessed 2026-09-14).
- [2] MDN Web Docs, "Math.random() — JavaScript," Mozilla Developer Network, 2025. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random (accessed 2026-09-14).
- [3] GitHub Docs, "About GitHub Pages," GitHub, Inc. <https://docs.github.com/en/pages/getting-started-with-github-pages/about-github-pages> (accessed 2026-09-14).
- [4] B. Schwartz, *The Paradox of Choice: Why More Is Less*. New York, NY, USA: HarperCollins, 2004.
- [5] Twitter/X, "Twemoji — Twitter Emoji (SVG), CC-BY 4.0," GitHub Repository. <https://github.com/jdecked/twemoji> (accessed 2026-09-14).
- [6] SVG Repo, "Free SVG Vectors," SVG Repo. <https://www.svgrepo.com> (accessed 2026-09-14).